

The `proton_pdf` Numerical Experiment

Computational HEP

1 Purpose of the experiment

This numerical experiment is a guided tour of proton parton distribution functions, usually called PDFs. It does three things:

1. it uses LHAPDF to plot $xf_i(x, \mu_F^2)$ for several parton flavours and several factorisation scales μ_F ;
2. it checks basic proton sum rules, such as momentum conservation and valence-quark number conservation;
3. it implements a deliberately small leading-order DGLAP evolution solver and compares its result to LHAPDF.

The most important pedagogical goal is not to produce the most accurate PDF evolution possible. The goal is to make the logic visible. We want to see what a PDF is, why it depends on a scale, what equation controls that scale dependence, and how one can solve that equation numerically.

2 What is a PDF?

A proton is not an elementary particle. It contains quarks, antiquarks, and gluons. In a high-energy collision, the hard scattering usually happens over a very short time and distance scale. During that hard scattering, a single parton from each proton participates directly. The rest of the proton becomes beam remnant.

The PDF $f_i(x, \mu_F)$ is a probability-density-like object. It tells us how many partons of type i we expect to find inside the proton with momentum fraction x , when the proton is probed at factorisation scale μ_F . The index i can be

$$g, \quad u, \quad \bar{u}, \quad d, \quad \bar{d}, \quad s, \quad \bar{s}, \quad \dots$$

The variable x is a fraction, so

$$0 < x < 1.$$

If a proton has momentum P^μ , then a parton carrying fraction x has approximately momentum

$$p^\mu \simeq xP^\mu.$$

Strictly speaking, PDFs are not ordinary probabilities. They are renormalised quantum field theory matrix elements. However, for the purposes of a first collider-physics calculation, it is very useful to think of them as number densities in x .

3 Why does LHAPDF return xf_i ?

LHAPDF's basic function is

$$\text{xfxQ}(i, x, Q).$$

It returns

$$xf_i(x, Q),$$

not $f_i(x, Q)$. This is common in PDF plotting because xf_i has a very direct interpretation: it is the momentum-density contribution from parton flavour i . If $f_i(x, Q)dx$ counts partons in the interval $[x, x + dx]$, then

$$xf_i(x, Q)dx$$

counts the fraction of proton momentum carried by those partons in that interval.

That is why the experiment plots xf_i . It is also why the momentum sum rule takes such a simple form.

4 The factorisation scale μ_F

The symbol μ_F is called the factorisation scale. It separates two kinds of physics:

- long-distance physics inside the proton, which is put into the PDFs;
- short-distance physics in the hard scattering, which is computed with Feynman diagrams.

This separation is called factorisation.

The split is artificial. Nature does not come with a label saying which gluon emission is “inside the PDF” and which gluon emission is “inside the hard matrix element”. We choose a scale μ_F , and we define emissions below that scale to be absorbed into the PDF. Emissions above that scale are treated as part of the hard scattering.

Because the split is artificial, a fully physical prediction should not depend on μ_F . In an all-orders calculation this cancellation is exact:

$$\frac{d}{d \ln \mu_F^2} [\text{PDFs at } \mu_F \otimes \text{hard scattering at } \mu_F] = 0.$$

At finite perturbative order, the cancellation is incomplete. That is why varying μ_F is often used to estimate missing higher-order corrections.

5 Where does PDF running come from?

PDFs depend on μ_F . This scale dependence is often called PDF evolution or PDF running. The word “running” is used by analogy with the running of couplings, such as $\alpha_s(Q)$.

The physical origin is collinear radiation. Consider a quark moving very fast along the proton direction. It can radiate a gluon at a very small angle:

$$q \rightarrow qg.$$

Because the emission is almost parallel to the original quark, it is hard to resolve as a separate parton unless the probe has enough resolution. A larger μ_F means a sharper probe. As μ_F increases, the probe resolves more of the proton's partonic substructure. A quark at low resolution may be seen, at higher resolution, as a quark plus a gluon. Similarly, a gluon may split:

$$g \rightarrow q\bar{q}, \quad g \rightarrow gg.$$

Therefore the number and momentum distributions of quarks and gluons change with the probing scale.

In perturbative QCD this change is calculable when μ_F is large enough that $\alpha_s(\mu_F)$ is small. The equation that describes the change is the DGLAP equation.

6 The DGLAP equation

The DGLAP equation is an evolution equation in the variable

$$t = \ln Q^2.$$

At leading order it reads

$$\frac{\partial f_i(x, Q)}{\partial \ln Q^2} = \frac{\alpha_s(Q)}{2\pi} \sum_j \int_x^1 \frac{dz}{z} P_{ij}(z) f_j\left(\frac{x}{z}, Q\right).$$

Let us go through this slowly.

- $f_i(x, Q)$ is the PDF for the parton we observe after evolution.
- $f_j(x/z, Q)$ is the PDF for the parent parton before the splitting.
- z is the momentum fraction retained by the observed daughter parton relative to its parent.
- $P_{ij}(z)$ is a splitting kernel. It gives the probability pattern for a parent parton j to produce a daughter parton i .
- The integral from x to 1 says that the parent must have had at least as much momentum as the daughter.

The experiment evolves

$$F_i(x, Q) \equiv x f_i(x, Q),$$

because this is what LHAPDF returns and what the plots show. Multiplying the DGLAP equation by x , and using

$$F_j\left(\frac{x}{z}, Q\right) = \frac{x}{z} f_j\left(\frac{x}{z}, Q\right),$$

one finds

$$\frac{\partial F_i(x, Q)}{\partial \ln Q^2} = \frac{\alpha_s(Q)}{2\pi} \sum_j \int_x^1 dz P_{ij}(z) F_j\left(\frac{x}{z}, Q\right).$$

Notice that the explicit $1/z$ has disappeared in the equation for $F = xf$. This is exactly the form implemented in the code.

7 The leading-order splitting kernels

The experiment uses the leading-order QCD splitting kernels. The colour factors for $SU(3)$ are

$$C_F = \frac{4}{3}, \quad C_A = 3, \quad T_R = \frac{1}{2}.$$

The number of active quark flavours in the experiment is

$$n_f = 4,$$

corresponding to d, u, s, c .

The kernels are

$$P_{qq}(z) = C_F \left[\frac{1+z^2}{1-z} \right]_+ + \frac{3}{2} C_F \delta(1-z),$$

$$P_{qg}(z) = T_R [z^2 + (1-z)^2],$$

$$P_{gq}(z) = C_F \frac{1 + (1-z)^2}{z},$$

and

$$P_{gg}(z) = 2C_A \left[\frac{z}{(1-z)_+} + \frac{1-z}{z} + z(1-z) \right] + \frac{\beta_0}{2} \delta(1-z),$$

with

$$\beta_0 = \frac{11C_A - 4T_R n_f}{3}.$$

The names mean the following:

- P_{qq} : a quark remains a quark after radiating a gluon;
- P_{qg} : a gluon produces a quark or antiquark through $g \rightarrow q\bar{q}$;
- P_{gq} : a quark radiates a gluon, so the observed daughter is the gluon;
- P_{gg} : a gluon remains a gluon after gluon radiation.

8 What is the plus prescription?

Some splitting kernels contain expressions like

$$\frac{1}{1-z}.$$

This becomes singular when $z \rightarrow 1$. Physically, $z \rightarrow 1$ corresponds to an emission in which the emitted parton is very soft. The real-emission singularity is cancelled by virtual corrections. In the compact DGLAP kernels, this cancellation is encoded by the plus prescription.

For a singular function $g(z)$, the plus distribution is defined by how it acts under an integral. If $h(z)$ is a smooth test function, then

$$\int_0^1 dz [g(z)]_+ h(z) = \int_0^1 dz g(z) [h(z) - h(1)].$$

The subtraction $h(z) - h(1)$ makes the integral finite at $z = 1$.

Our convolution does not start at 0. It starts at x :

$$\int_x^1 dz [g(z)]_+ h(z).$$

The code uses the identity

$$\int_x^1 dz [g(z)]_+ h(z) = \int_x^1 dz g(z) [h(z) - h(1)] - h(1) \int_0^x dz g(z).$$

For the P_{qq} singular piece,

$$g_{qq}(z) = C_F \frac{1+z^2}{1-z},$$

the primitive needed for the second term is

$$\int_0^x dz g_{qq}(z) = C_F \left[-2 \ln(1-x) - x - \frac{x^2}{2} \right].$$

For the plus part of P_{gg} ,

$$g_{gg}(z) = 2C_A \frac{z}{1-z},$$

the primitive is

$$\int_0^x dz g_{gg}(z) = 2C_A [-\ln(1-x) - x].$$

These formulae are implemented explicitly. This is a very useful part of the exercise because most production PDF-evolution codes hide the plus-prescription bookkeeping from the user.

9 The evolution system solved by the code

The state vector contains

$$F_g(x), \quad F_q(x), \quad F_{\bar{q}}(x)$$

for the active flavours $q = d, u, s, c$. At a fixed x , the quark equation is schematically

$$\frac{\partial F_q}{\partial \ln Q^2} = \frac{\alpha_s}{2\pi} [P_{qq} \otimes F_q + P_{qg} \otimes F_g],$$

and the antiquark equation is the same:

$$\frac{\partial F_{\bar{q}}}{\partial \ln Q^2} = \frac{\alpha_s}{2\pi} [P_{q\bar{q}} \otimes F_{\bar{q}} + P_{qg} \otimes F_g].$$

The gluon equation is

$$\frac{\partial F_g}{\partial \ln Q^2} = \frac{\alpha_s}{2\pi} \left[P_{gg} \otimes F_g + \sum_q P_{gq} \otimes (F_q + F_{\bar{q}}) \right].$$

Here $\otimes$ denotes the convolution

$$(P \otimes F)(x) = \int_x^1 dz P(z) F\left(\frac{x}{z}\right).$$

10 Discretising the x dependence

The DGLAP equation is not an ordinary differential equation in one variable. It is an integro-differential equation. The derivative is with respect to $\ln Q^2$, but the right-hand side contains an integral over z and the PDF evaluated at shifted points x/z .

The experiment turns it into a finite-dimensional ODE problem as follows.

1. Choose an x -grid:

$$x_1, x_2, \dots, x_N.$$

The code uses a hybrid grid, with many points at small x and a linear tail toward large x . This is because PDFs often change very quickly at small x , while valence structure is important at moderate x .

2. Store the values

$$F_i(x_a, Q)$$

for every parton flavour i and every grid point x_a .

3. When the convolution asks for

$$F_i(x_a/z, Q),$$

the point x_a/z will usually not lie exactly on the grid. The code therefore uses linear interpolation.

4. At $x = 1$, the PDF is forced to zero. This is physically sensible: a single parton carrying exactly all of the proton momentum has zero density in the continuous distribution.
5. The z -integrals are evaluated by a trapezoidal quadrature.

After these steps, the DGLAP equation becomes a large but ordinary system:

$$\frac{d\mathbf{F}}{dt} = \mathbf{R}(\mathbf{F}, t), \quad t = \ln Q^2.$$

The vector $\mathbf{F}$ contains all parton flavours at all x -grid points. The function $\mathbf{R}$ is the right-hand side built from the splitting kernels and convolutions.

11 The Runge–Kutta scale evolution

To evolve from Q_0 to Q_1 , the code evolves in

$$t = \ln Q^2.$$

The starting and ending values are

$$t_0 = \ln Q_0^2, \quad t_1 = \ln Q_1^2.$$

If the user requests N_{step} evolution steps, then

$$\Delta t = \frac{t_1 - t_0}{N_{\text{step}}}.$$

The code uses the fourth-order Runge–Kutta method. For an ODE

$$\frac{d\mathbf{F}}{dt} = \mathbf{R}(\mathbf{F}, t),$$

one step is

$$\begin{aligned} \mathbf{k}_1 &= \mathbf{R}(\mathbf{F}_n, t_n), \\ \mathbf{k}_2 &= \mathbf{R}\left(\mathbf{F}_n + \frac{\Delta t}{2}\mathbf{k}_1, t_n + \frac{\Delta t}{2}\right), \\ \mathbf{k}_3 &= \mathbf{R}\left(\mathbf{F}_n + \frac{\Delta t}{2}\mathbf{k}_2, t_n + \frac{\Delta t}{2}\right), \\ \mathbf{k}_4 &= \mathbf{R}(\mathbf{F}_n + \Delta t \mathbf{k}_3, t_n + \Delta t), \end{aligned}$$

and

$$\mathbf{F}_{n+1} = \mathbf{F}_n + \frac{\Delta t}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4).$$

This is a standard ODE method. It is not specific to QCD. The QCD content is inside the right-hand side $\mathbf{R}$.

12 Why the RK4 formula is reasonable

It is useful to justify the Runge–Kutta formula, because otherwise it can look like a mysterious recipe. The starting point is the exact integral form of the ODE. If

$$\frac{d\mathbf{F}}{dt} = \mathbf{R}(\mathbf{F}, t),$$

then over one small step,

$$\mathbf{F}(t_n + \Delta t) = \mathbf{F}(t_n) + \int_{t_n}^{t_n + \Delta t} dt \mathbf{R}(\mathbf{F}(t), t).$$

So the whole problem is this: we need the average value of the slope $\mathbf{R}$ across the interval, but we do not know the exact curve $\mathbf{F}(t)$ inside the interval.

The simplest method, Euler’s method, uses only the slope at the beginning:

$$\mathbf{F}_{n+1} \approx \mathbf{F}_n + \Delta t \mathbf{R}(\mathbf{F}_n, t_n).$$

This is easy to understand, but it is crude. It assumes that the slope at the left endpoint is representative of the whole interval. If the slope changes noticeably during the step, Euler’s method misses that change.

RK4 improves this by sampling the slope four times:

- $\mathbf{k}_1$ is the slope at the beginning of the step;
- $\mathbf{k}_2$ is an estimate of the slope halfway through the step, using $\mathbf{k}_1$ to guess the midpoint value of $\mathbf{F}$;
- $\mathbf{k}_3$ is another midpoint slope, now using the better midpoint estimate from $\mathbf{k}_2$;
- $\mathbf{k}_4$ is an estimate of the slope at the end of the step, using $\mathbf{k}_3$ to guess the endpoint value of $\mathbf{F}$.

The final combination

$$\frac{1}{6} (\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)$$

is a weighted average slope. The midpoint slopes receive twice the weight of the endpoint slopes because they usually give a better estimate of the average slope across the interval. This is closely related in spirit to Simpson's rule for numerical integration, where midpoint information greatly improves the estimate of an integral.

One can make this statement precise by expanding the exact solution in a Taylor series in Δt . RK4 is constructed so that its Taylor expansion agrees with the exact solution through terms of order $(\Delta t)^4$. Therefore the error made in a single step is of order

$$(\Delta t)^5,$$

and after many steps over a fixed interval the accumulated error is of order

$$(\Delta t)^4.$$

That is why it is called a fourth-order method. For this exercise, RK4 is a good compromise: it is much more accurate than Euler's method, but it is still short enough to write explicitly and inspect line by line.

At every intermediate RK4 evaluation, the code converts the current t back to a scale

$$Q = e^{t/2}$$

and asks LHAPDF for $\alpha_s(Q)$. Thus the pedagogical DGLAP solver uses LHAPDF's running coupling but computes the PDF evolution itself.

The plot produced by the experiment now shows two RK4 choices. The first is the user-configured evolution in equal steps of $t = \ln Q^2$. The second is a fixed comparison evolution in ten equal steps of Q itself. For the default comparison

$$Q_0 = 2 \text{ GeV}, \quad Q_1 = 10 \text{ GeV},$$

the second path uses

$$Q = 2.0, 2.8, 3.6, \dots, 10.0 \text{ GeV}.$$

Each of these intervals is still evolved with RK4 in the variable $t = \ln Q^2$; only the way the interval is subdivided is different. Showing both curves is a useful numerical check: if the step size is small enough and the implementation is stable, changing the subdivision should not drastically change the result. The bottom panel of the comparison plot makes this explicit by showing

$$\log_{10} \left(\frac{|F_{\text{uniform } \ln Q^2}(x) - F_{\text{linear } Q}(x)|}{|F_{\text{LHAPDF}}(x, Q_1)|} \right),$$

so that two visually overlapping curves can still be checked quantitatively.

13 Where Symbolica enters

The code uses Symbolica to build readable symbolic expressions for the splitting kernels. For example, the singular part of P_{qq} is represented symbolically as

$$C_F \frac{1+z^2}{1-z}.$$

The numerical quadrature then uses fast vectorised functions corresponding to those symbolic expressions. This gives a useful compromise: the code contains explicit mathematical formulae for the kernels, but the default run is still fast enough for a classroom exercise.

14 Comparison with LHAPDF

The DGLAP comparison proceeds as follows:

1. choose a starting scale Q_0 ;
2. query LHAPDF for $F_i(x, Q_0) = x f_i(x, Q_0)$ on the x -grid;
3. evolve those values to a target scale Q_1 with the pedagogical LO solver, once with the configured equal- $\ln Q^2$ RK4 steps and once with ten equal- Q RK4 steps;
4. query LHAPDF directly at Q_1 ;
5. plot the starting LHAPDF shape, the target LHAPDF shape, both evolved answers, and the relative differences.

The comparison plot is named

`proton_pdf_dglap_lo_vs_lhapdf.png`.

The evolved curves should not be expected to agree perfectly with LHAPDF. The comparison is not a precision validation of a full PDF-evolution program. Differences arise because:

- the experiment uses leading-order splitting kernels;
- the default LHAPDF set is an NLO fitted PDF set;
- the x -grid is coarse compared with production codes;
- interpolation and trapezoidal quadrature are simple approximations;
- heavy-flavour threshold treatment is simplified;
- only the active flavours d, u, s, c are evolved in the toy solver.

The point is to see the mechanism of PDF running, not to reproduce LHAPDF's internal machinery exactly.

15 Momentum and valence sum rules

The experiment also checks several sum rules. These are conservation laws or proton quantum-number statements written in PDF language.

15.1 Momentum sum rule

All partons together carry all of the proton momentum. Therefore

$$\int_0^1 dx \left[xg(x, Q) + \sum_q xq(x, Q) + \sum_q x\bar{q}(x, Q) \right] = 1.$$

Because LHAPDF returns xf_i , the integrand is directly the sum of LHAPDF queries:

$$xg + xq + x\bar{q}.$$

The code performs this integral numerically on the x -grid.

15.2 Valence sum rules

The proton contains two valence up quarks and one valence down quark. Sea quarks come in quark-antiquark pairs and therefore do not contribute to valence number. The valence distributions are

$$\begin{aligned} u_v(x, Q) &= u(x, Q) - \bar{u}(x, Q), \\ d_v(x, Q) &= d(x, Q) - \bar{d}(x, Q). \end{aligned}$$

The sum rules are

$$\int_0^1 dx u_v(x, Q) = 2, \quad \int_0^1 dx d_v(x, Q) = 1.$$

For strange and charm in this simplified check,

$$\int_0^1 dx s_v(x, Q) = 0, \quad \int_0^1 dx c_v(x, Q) = 0.$$

Here the code must divide LHAPDF's xq by x , because the valence number integral is over $q - \bar{q}$, not over $x(q - \bar{q})$:

$$q_v(x, Q) = \frac{xq(x, Q) - x\bar{q}(x, Q)}{x}.$$

15.3 Baryon number

Each quark carries baryon number $1/3$. Antiquarks carry baryon number $-1/3$. The proton has baryon number 1. Therefore

$$B = \frac{1}{3} \int_0^1 dx [u_v(x, Q) + d_v(x, Q) + s_v(x, Q) + c_v(x, Q)] \simeq 1.$$

15.4 Isospin

For the proton, the third component of isospin is

$$I_3 = +\frac{1}{2}.$$

In this simplified parton-counting picture,

$$I_3 = \frac{1}{2} \left[\int_0^1 dx u_v(x, Q) - \int_0^1 dx d_v(x, Q) \right] \simeq \frac{1}{2}.$$

16 Numerical integration of the sum rules

The sum rules are integrals over $x \in [0, 1]$. The code cannot evaluate a continuum integral exactly. It samples a finite x -grid and uses the trapezoidal rule:

$$\int_a^b dx F(x) \approx \sum_{k=1}^{N-1} \frac{x_{k+1} - x_k}{2} [F(x_k) + F(x_{k+1})].$$

The resulting numbers are written to

`proton_pdf_sum_rules.csv`.

Small deviations from the exact targets can come from grid truncation, grid coarseness, and interpolation details in the PDF set.

17 Example command

A typical run is

```
./run.py --proton_pdf \  
  --pdf_set NNPDF40_nlo_as_01180_nf_4 \  
  --q_values 2.0,10.0,91.1876 \  
  --flavours g,u,d,s,c,ubar,dbar,sbar,cbar \  
  --dglap_start_q 2.0 \  
  --dglap_target_q 10.0 \  
  --dglap_flavour u \  
  --dglap_x_points 45 \  
  --dglap_z_points 80 \  
  --dglap_steps 4 \  
  --output_dir proton_pdf_lo_run
```

The output directory contains:

`proton_pdf_xfx_Q*.png`,
`proton_pdf_sum_rules.csv`,
`proton_pdf_dglap_lo_vs_lhapdf.png`.

18 What to learn from this experiment

The main lessons are:

- PDFs describe the proton's partonic content as a function of momentum fraction x and scale Q .
- The scale dependence is not arbitrary. It is caused by resolvable collinear QCD radiation.
- DGLAP evolution turns this physical idea into an equation.
- The equation is non-local in x : the PDF at x receives contributions from parent partons at x/z .
- The plus prescription is the mathematical way real and virtual soft singularities combine into a finite evolution equation.
- Numerical PDF evolution requires choices: an x -grid, interpolation, quadrature, a running coupling, and an ODE solver.
- Sum rules are powerful checks that the PDFs still describe a proton.